Description of MATLAB Structured-Light Extraction Programs

-- Code Snippet Explanation and Experimental Result Images

This document is prepared based on the MATLAB programs used in the experiment. It briefly explains the function of each key code segment, the basic operating logic, and the corresponding experimental result images. Only representative code snippets are selected, rather than the full program, so that the document can be used for presentation, video recording, or program explanation.

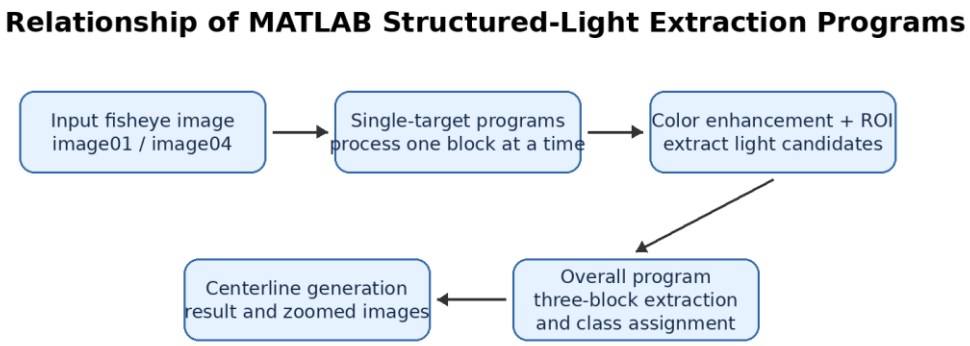

Figure 1 Relationship of MATLAB structured-light extraction programs

1 Overall Relationship of the Programs

The experimental programs can be divided into two types. The first type is the single-target structured-light extraction program, which is used to verify whether the structured light on each wooden block can be extracted separately. The second type is the multi-target synchronous extraction program, which identifies three small wooden blocks in the same image and extracts the structured-light centerline on each block surface.

Program type	Main input	Core function	Main output
First-block single-target program	image01.jpg	Extract the red laser line inside the first black-natural-color wooden block area	ROI image, candidate-region image, black-background line image, detection result image
Second-block single-target program	image01.jpg	Extract the magenta laser line inside the blue surface of the second red-blue block	Blue-region image, magenta candidate image, detection result image
Third-block extraction logic	image01 / image04	Apply line-shaped filtering and centerline fitting to the red structured light on the gray-purple block	Third-block detection result image and local zoomed image
Three-block synchronous program	image04.jpg + trained net	Perform semantic segmentation first, then use semantic regions to filter global structured-light candidates	Class-assignment result, three centerlines, endpoint coordinates, and running time

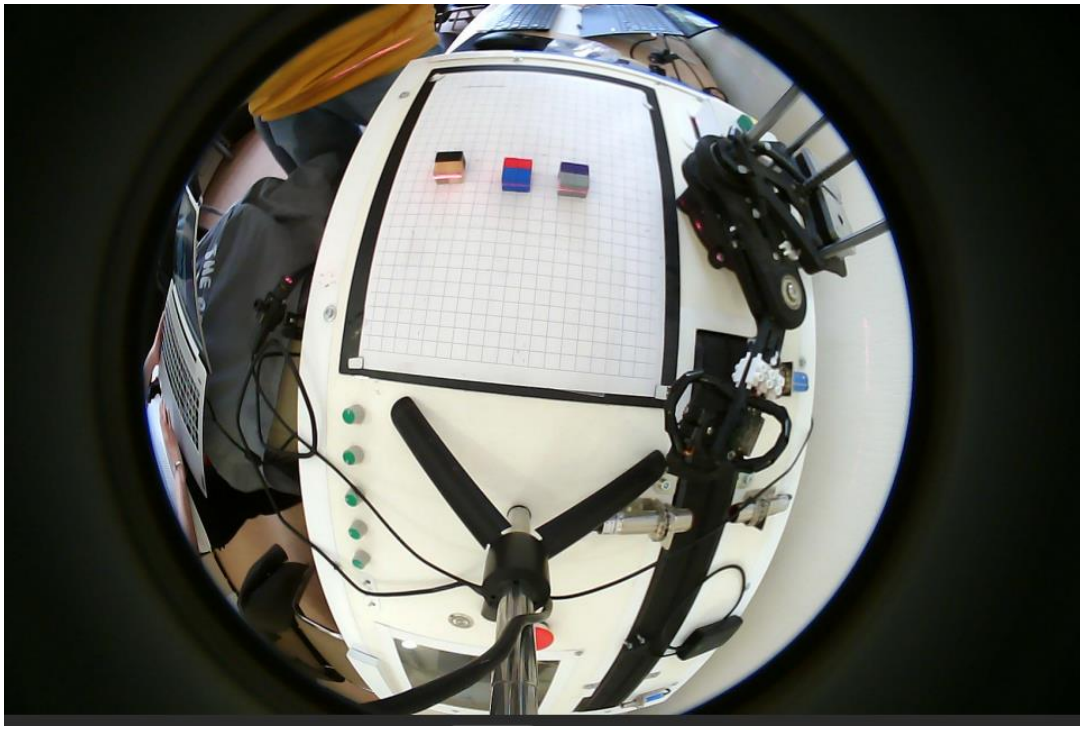

Figure 2 Original experimental image: three small blocks and the experimental platform in one fisheye image

2 Single-Target Structured-Light Extraction Program for the First Block

The first program mainly processes the black-natural-color two-tone wooden block. The structured light on this block mainly falls on the natural-color surface, and the laser color is relatively clear. Therefore, the program uses a fixed ROI, red-color enhancement, row projection, and horizontal-line output to complete the structured-light extraction.

2.1 Image Reading and Color-Space Conversion

```
imgName = 'image01.jpg';
black_image = imread(imgName);
img = im2double(black_image);

R = img(:,:,1);
G = img(:,:,2);
B = img(:,:,3);

hsvImage = rgb2hsv(img);
H = hsvImage(:,:,1);
S = hsvImage(:,:,2);
V = hsvImage(:,:,3);
```

This code reads the experimental image and decomposes it into RGB and HSV color information. The RGB channels are used to judge whether the red laser is obvious, while the HSV color space helps determine hue, saturation, and brightness, making the subsequent red-laser filtering more stable.

2.2 Setting the ROI and Laser Search Region of the First Block

```
x1 = round(0.375 * imgW);
x2 = round(0.455 * imgW);
y1 = round(0.200 * imgH);
```

```
y2 = round(0.315 * imgH);

laserY1 = round(y1 + 0.25 * (y2 - y1));
laserY2 = round(y1 + 0.60 * (y2 - y1));
laserMask(laserY1:laserY2, x1:x2) = true;
```

This segment first determines the approximate ROI of the first wooden block, and then further reduces the ROI to the laser search region. In this way, the program only searches for the laser near the small block, avoiding false detections from the robot arm, fisheye edge, or background reflection.

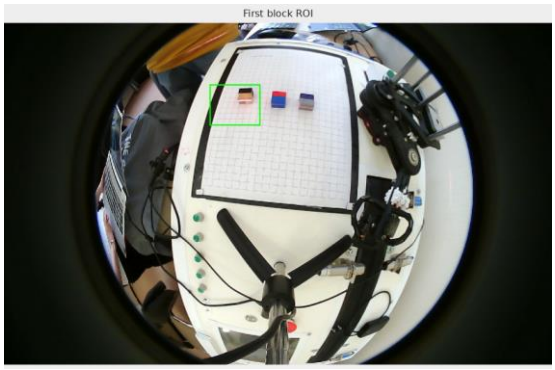

Figure 3 ROI of the first small block

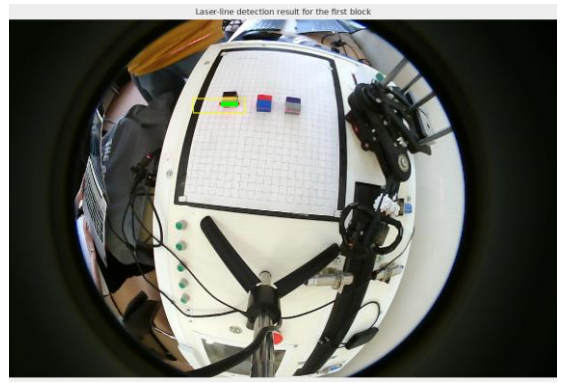

Figure 4 Structured-light detection result of the first block

2.3 Red-Laser Enhancement and Candidate Region Generation

```
redHSV = ((H >= 0.000 & H <= 0.090) | (H >= 0.900 & H <= 1.000)) & ...
         (S > 0.10) & (V > 0.20);

redRGB = (R > 0.18) & (R > 1.01 * G) & (R > 1.01 * B);
redScore = R - max(G, B);
redScore(redScore < 0) = 0;
redScore(~laserMask) = 0;

candidateMask = laserMask & redHSV & redRGB & (redScoreSmooth > 0.012);
candidateMask = bwareaopen(candidateMask, 1);
candidateMask = imclose(candidateMask, strel('line', 5, 0));
```

This part is the core of the first-block program. The program simultaneously uses the HSV red range, RGB red dominance, and red score to select candidate laser pixels. Area filtering and morphological closing are then used to remove small noise points and make the candidate region closer to a continuous laser line.

2.4 Locating the Center Row by Row Projection and Outputting the Final Segment

```
rowProfile = sum(scoreOnlyROI(:, x1:x2), 2);
rowProfileSmooth = movmean(rowProfile, 5);
[~, maxIdx] = max(rowProfileSmooth(roiRows));
bestRow = roiRows(maxIdx) + manualYOffset;

xDraw = xStart:xEnd;
yDraw = bestRow * ones(size(xDraw));
plot(xDraw, yDraw, 'g-', 'LineWidth', 7);
```

Row projection is used to find the horizontal position with the strongest red response. Since the laser on the first block is basically horizontal, the final centerline is directly represented by a green horizontal line and displayed on both the original image and the local zoomed image.

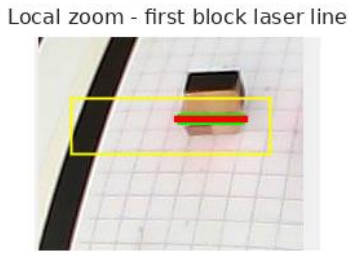

Figure 5 Local zoomed detection effect of the first block

3 Single-Target Structured-Light Extraction Program for the Second Block

The second program processes the red-blue two-tone wooden block. The real structured light on this target mainly appears on the blue surface, and its color is a weak magenta. If a normal red threshold is used, the red upper surface is easily misclassified as laser. Therefore, the program first locates the blue surface and then extracts the magenta structured light inside it.

3.1 Automatically Locating the Blue Region of the Second Block

```
blueMask = (H > 0.52 & H < 0.72) & ...
           (S > 0.15) & (V > 0.10) & ...
           (B > 0.15) & (B > 1.02 * R) & (B > 1.02 * G) & searchMask;

cc = bwconncomp(blueMask);
stats = regionprops(cc, 'Area', 'BoundingBox', 'Centroid');
[~, idx] = max(scores);
bbox = stats(idx).BoundingBox;
```

This code first searches for the blue region according to HSV and RGB conditions, and then uses connected components and position scoring to select the blue surface most likely belonging to the second block. This prevents the program from selecting other blue or dark regions by mistake.

3.2 Constructing the Laser Search Region Inside the Blue Surface

```
laser_x1 = max(floor(bx - 0.10 * bw), 1);
laser_x2 = min(ceil(bx + 1.10 * bw), imgW);
laser_y1 = max(floor(by + laser_y1_factor * bh), 1);
laser_y2 = min(ceil(by + laser_y2_factor * bh), imgH);

laserROIMask(laser_y1:laser_y2, laser_x1:laser_x2) = true;
```

This segment restricts the detection range to the blue surface. Since the laser is not on the red upper surface but on the blue surface, this ROI setting greatly reduces false detection.

3.3 Magenta-Laser Enhancement

```
R_bg = medfilt2(R, [15 15], 'symmetric');
localRed = R - R_bg;
localRed(localRed < 0) = 0;

sumRGB = R + G + B + eps;
rNorm = R ./ sumRGB;
gNorm = G ./ sumRGB;
bNorm = B ./ sumRGB;
```

```
purpleScore = 0.55 * rNorm + 0.35 * bNorm - 0.90 * gNorm + 1.20 * localRed;
purpleScore(~laserROIMask) = 0;
```

The blue background makes the B channel strong, so it is not reasonable to simply require R to be greater than B. Here, a magenta score is constructed by enhancing R, retaining B, and suppressing G, so that the weak magenta laser on the blue surface can be highlighted.

3.4 Generating the Final Laser Line

```
rowProfile = sum(scoreOnlyROI(:, laser_x1:laser_x2), 2);
[~, maxIdx] = max(rowProfileSmooth(roiRows));
bestRow = roiRows(maxIdx) + manualYOffset;

if length(xPoints) >= 3
    bLaser = median(yPoints - kBlue * xPoints);
else
    bLaser = bestRow - kBlue * centerX;
end

xDraw = laser_x1:laser_x2;
yDraw = round(kBlue * xDraw + bLaser);
```

The program first uses row projection to locate the most obvious magenta laser position, and then generates the final line segment according to the slope of the blue surface. The resulting line is basically parallel to the blue surface and better matches the actual laser stripe.

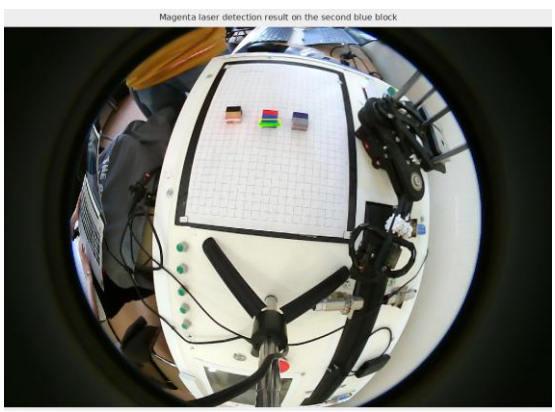

Figure 6 Structured-light detection result of the second block

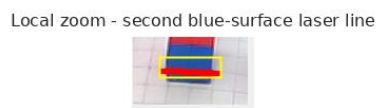

Figure 7 Local zoomed detection effect of the second block

4 Structured-Light Extraction Logic for the Third Block

The structured-light color on the third gray-purple block is relatively clear, but the target region is darker. Its processing idea is consistent with the first two blocks: determine the target region, extract structured-light candidate points, and then obtain the result through line-shaped filtering and centerline fitting. In the overall program, the purple target mainly enters the general candidate-filtering and PCA-fitting branch.

4.1 Candidate-Point Filtering and Line-Shape Judgment

```
localCandidate = laserCandidateAll & localAllowed;
vals = laserScore(localCandidate);
localThreshold = prctile(vals, 70);
localLaser = localCandidate & (laserScore >= localThreshold);
```

```
stats = regionprops(cc, 'Area', 'BoundingBox', 'PixelIdxList', ...
                    'MajorAxisLength', 'MinorAxisLength', 'Orientation');

lineRatio = majorLen / minorLen;
lineOK = lineRatio >= 1.10;
```

This code filters structured-light candidate points inside the current small-block region and checks whether the candidate region has a thin line-shaped feature. A larger ratio between the major axis and the minor axis means that the region is closer to a laser line.

4.2 Generating the Centerline with PCA

```
points = [xBest, yBest];
centerPoint = mean(points, 1);
pointsZero = points - centerPoint;

[~, ~, Vpca] = svd(pointsZero, 'econ');
direction = Vpca(:, 1)';

xLine = centerPoint(1) + linspace(tLine1, tLine2, 100) * direction(1);
yLine = centerPoint(2) + linspace(tLine1, tLine2, 100) * direction(2);
```

PCA fitting is used to generate the centerline according to the overall distribution direction of the candidate points. For the slightly inclined structured light on the gray-purple block, PCA is more suitable than forcing a horizontal line.

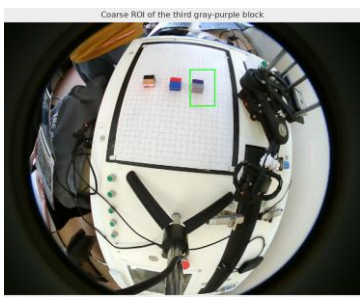

Figure 8 ROI of the third block

Figure 9 Structured-light detection result of the third block

Figure 10 Local zoomed detection effect of the third block

5 Overall Program for Synchronous Extraction of Three Blocks

The purpose of the overall program is to identify three small blocks in the same laser-illuminated image and extract the structured-light centerline on each block surface. The main difference from the single-target programs is that the single-target programs mainly rely on manual or fixed ROIs, while the overall program first performs semantic segmentation and then uses semantic regions to constrain the structured-light candidate regions, thus realizing class assignment.

5.1 Loading the Model and Performing Semantic Segmentation

```
netFile = '3DtrainedNetresnet50-50.mat';
load(netFile);
I = imresize(I0, inputSize);

C = semanticseg(I, net, 'MiniBatchSize', 1);
B_overlay = labeloverlay(I, C, 'Colormap', cmap, 'Transparency', 0.45);
```

This part first loads the trained DeepLabV3+ semantic segmentation model and then performs pixel-level classification on the input image. According to the segmentation result, the program obtains the target regions of the black, red, and purple small blocks.

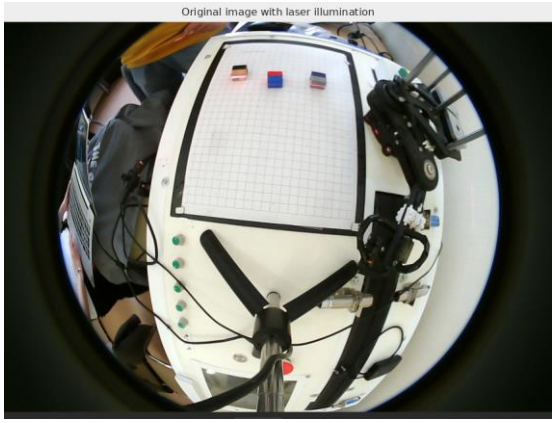

Figure 11 Original laser-illuminated image input to the overall program

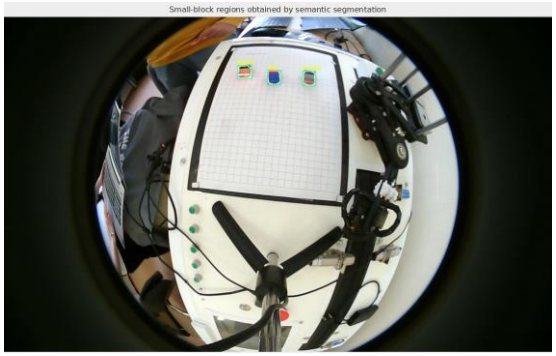

Figure 12 Three small-block regions obtained by semantic segmentation

5.2 Extracting Target Masks from Semantic Segmentation Results

```
objects = buildObjectsFromSegmentation(C, targetClasses, sceneMask, keepLargestObjectPerClass);

classMask = (C == className);
classMask = bwareaopen(classMask, 20);
classMask = imclose(classMask, strel('disk', 5));
classMask = imfill(classMask, 'holes');
```

The semantic segmentation result may contain small noise points or broken boundaries. Therefore, the program applies area filtering, morphological closing, and hole filling to each target region. The processed mask is then used to limit the subsequent structured-light detection range.

5.3 Global Structured-Light Candidate Region Extraction

```
[laserScore, laserCandidateAll] = buildGlobalLaserCandidate( ...
    R, G, B, H, S, V, sceneMask, laserMode);

colorCandidate = redHSV | redRGB | magentaCandidate | purpleOnBlueCandidate;
rawCandidate = redRaw | brightRaw;
candidateMask = rawCandidate & (laserScoreSmooth > threshold);
```

This part searches the whole image for all possible structured-light regions. Because the full image contains reflections from the robot arm, edge highlights, and table reflections, the candidate region may include many non-target areas and cannot be used directly as the final result.

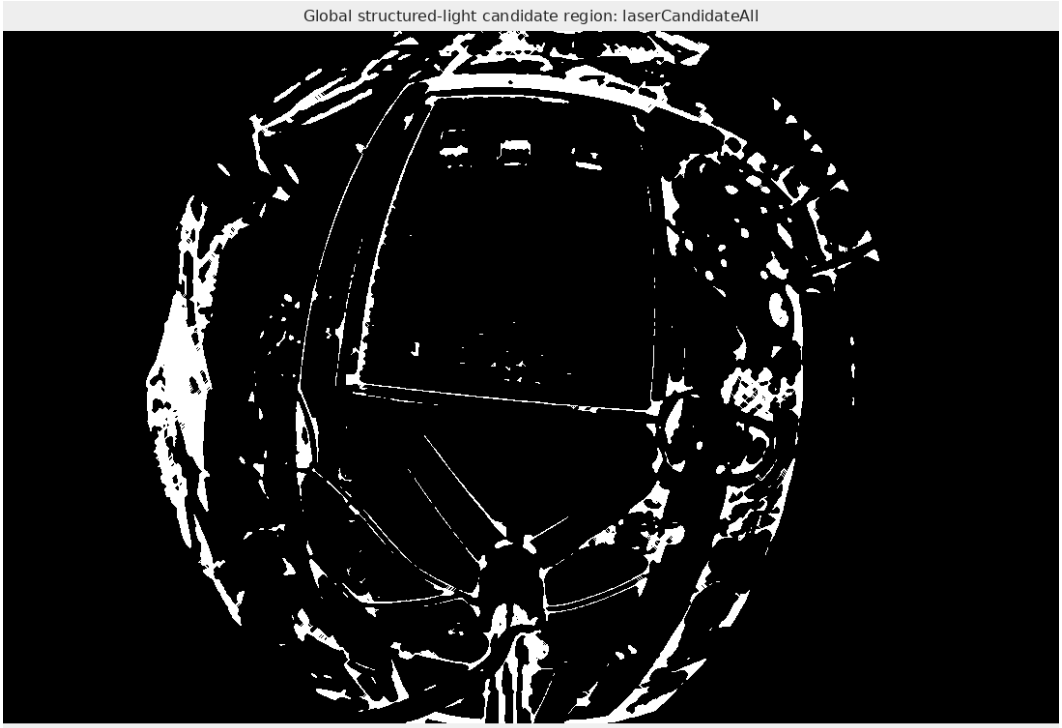

Figure 13 Global structured-light candidate region: many background interferences are also included

5.4 Filtering Candidate Structured Light with Small-Block Regions

```
results = extractLaserForEachObject( ...
    objects, laserCandidateAll, laserScore, objectSearchDilateRadius);

objSearch = imdilate(objMask, strel('disk', searchDilateRadius));
localCandidate = laserCandidateAll & localAllowed;
```

This is the key logic of the overall program. The program intersects the global structured-light candidates with the target region of each small block, retaining only the light stripes near the small blocks. In this way, interferences from the robot arm, black border, and table reflection can be removed, and each structured-light line can be assigned to the correct block.

5.5 Outputting the Final Detection Results and Running Time

```
plot(results(i).xLine, results(i).yLine, 'g-', 'LineWidth', 3);

fprintf('Semantic segmentation time: %.4f s\n', elapsedTime);
fprintf('Global structured-light candidate extraction time: %.4f s\n', candidateTime);
fprintf('Small-block structured-light centerline extraction time: %.4f s\n', centerlineTime);
fprintf('Total structured-light extraction time: %.4f s\n', laserTotalTime);
fprintf('Total program running time: %.4f s\n', programTotalTime);
```

Finally, the program draws the structured-light centerline of each small block on the original image and outputs the black-background centerline image and running time. The original-image result is used to check whether the structured light falls on the correct block, while the black-background image makes it easier to check the position and number of the three centerlines.

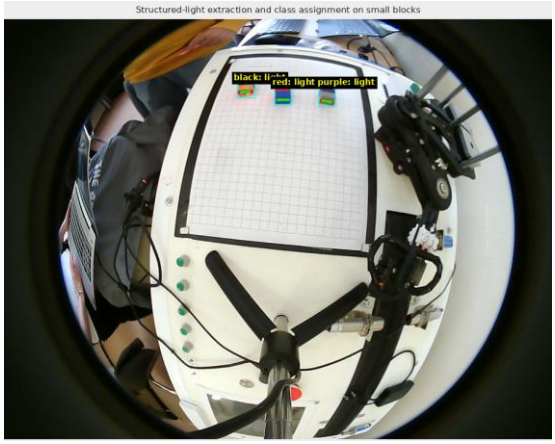

Figure 14 Synchronous structured-light extraction and class-assignment result for three blocks

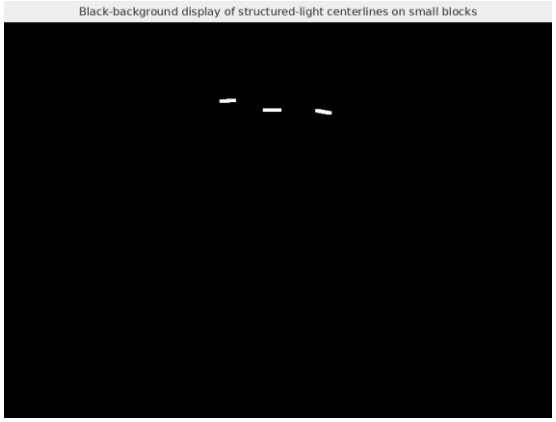

Figure 15 Black-background display of the structured-light centerlines on three blocks

6 Brief Summary

From the program logic, the single-target programs are mainly used to verify whether the structured light on one small block can be accurately extracted. The overall program then adds semantic segmentation, allowing the program to automatically distinguish the black, red, and purple small blocks and extract their corresponding structured-light centerlines.

The key point of the first-block program is fixed ROI and red-laser enhancement. The key point of the second-block program is to identify the blue surface first and then extract the magenta laser on that surface. The key point of the third-block program is line-shaped filtering and PCA fitting for the structured light on the gray-purple block. The key point of the overall program is to use semantic segmentation results to constrain global structured-light candidates, thereby reducing false detections in complex backgrounds.